

Capitolo 7b - Sincronizzazione all'interno del kernel

Status Done

Quando i thread vengono eseguiti nel kernel del SO, si verificano delle sfide di sincronizzazione spesso a causa della presenza di architetture multiprocessore (SMP) o del rischio di interruzioni e prelazioni.

Per questo motivo, i SO come Windows e Linux forniscono meccanismi di sincronizzazione specifici per l'uso esclusivo del kernel, in aggiunta a quelli offerti agli sviluppatori di applicazioni in modalità utente. I metodi di sincronizzazione forniti variano, ma le implementazioni più comuni nel kernel sono gli spinlock, i lock mutex e le tecniche che controllano le interruzioni e la prelazione.

Sincronizzazione in Windows

Il kernel di windows è multithread e supporta sistemi multiprocessore e applicazioni real-time.

- **Sistemi monoprocesore:** nelle architetture single-core, windows protegge le risorse globali del kernel disabilitando temporaneamente le interruzioni.
- **Sistemi multiprocessore:** negli ambienti SMP, windows utilizza **semafori ad attesa attiva (spinlock)** per proteggere le strutture dati del kernel. Windows utilizza gli spinlock solo per proteggere le strutture dati che richiedono una protezione di breve durata.
- **Oggetti dispatcher:** windows fornisce oggetti di sincronizzazione a livello di kernel noti come oggetti dispatcher (es. mutex e semafori). Questi oggetti permettono ai thread di bloccarsi (entrare in stato di attesa) se la risorsa è già in uso.
- **Mutex leggeri:** windows utilizza anche oggetti di sincronizzazione "leggeri" in modalità utente che si basano sul kernel solo in caso di contesa. Questo li rende più efficienti per gli sviluppatori di applicazioni, poiché il kernel (più costoso da usare) viene coinvolto solo quando strettamente necessario.

Sincronizzazione dei processi in Linux

Il kernel di Linux fornisce diversi meccanismi per la sincronizzazione interna, sviluppati con l'evoluzione del sistema da non prelazionabile a prelazionabile-

- **Interi atomici:** la tecnica più semplice è l'uso di interi atomici (`atomic_t`) che forniscono operazioni atomiche (indivisibili) su tipi di dati base, come l'incremento di un contatore. Sono usati dove è necessaria una mutua esclusione su una singola variabile.
- **Spinlock:** sono il meccanismo principale per la protezione delle sezioni critiche brevi.
 - Sui sistemi monoprocesore, uno spinlock viene implementato semplicemente disabilitando le interruzioni.
 - Sui sistemi multiprocessore, uno spinlock è un meccanismo che mette il thread in attesa attiva (busy waiting) finché il lock non è disponibile.
- **Mutex:** sono usati per proteggere strutture dati che richiedono una protezione più a lungo termine, dove l'attesa attiva degli spinlock sarebbe inefficiente.
- **Controllo della prelazione nel kernel:**
 - Linux implementa la prelazione (preemption) con le chiamate `preempt_disable()` e `preempt_enable()` .
 - La prelazione è disabilitata nel kernel se un task detiene un lock. Il kernel tiene traccia dei lock attivi per ciascun task con un contatore (`preempt_count`).
 - La prelazione è possibile solo se il contatore è a zero e le interruzioni non sono state esplicitamente disabilitate.

L'uso combinato di spinlock e disabilitazione della prelazione/interrupt permette a Linux di proteggere efficacemente le sue strutture dati critiche in tutte le configurazioni hardware.

Sincronizzazione POSIX

L'API POSIX (Pthreads) offre un insieme di strumenti per la sincronizzazione dei thread a livello utente, che include lock mutex, semafori e variabili condizionali. Questi strumenti sono implementati utilizzando in ultima analisi le primitive di sincronizzazione fornite dal SO ospite.

Lock Mutex POSIX

I lock mutex sono il meccanismo di sincronizzazione fondamentale in ambiente Pthreads.

- **Scopo:** proteggere le sezioni critiche del codice, assicurando l'esclusione mutua tra i thread.
- **Funzionamento:** un thread deve acquisire il lock prima di entrare nella sezione critica e rilasciarlo quando ne esce. Se il lock è già detenuto, il thread richiedente viene bloccato in attesa.
- **Funzioni chiave:**
 - `pthread_mutex_lock()` : acquisisce il lock
 - `pthread_mutex_unlock()` : rilascia il lock
 - `pthread_mutex_init()` : inizializza il mutex

Semafori POSIX

I semafori in Pthreads, come quelli generici, sono variabili di conteggio usate per la sincronizzazione. A differenza dei mutex, possono essere usati per sincronizzare attività più complesse del semplice accesso esclusivo.

Semafori con nome

I semafori con nome sono identificati da un nome univoco nel SO, rendendoli facilmente accessibili a più processi non correlati.

- **Creazione/Accesso:** `sem_open()` è usata per creare un nuovo semaforo o accedere a uno esistente, specificandone il nome, i flag e il valore iniziale.
- **Vantaggio:** la denominazione permette a processi che non condividono la stessa memoria di sincronizzarsi, semplicemente facendo riferimento al nome comune.

Semafori senza nome

I semafori senza nome non hanno un nome globale.

- **Condivisione:** per essere condivisi tra processi, devono essere collocati in una regione di memoria condivisa che i processi possono mappare. Sono più semplici da usare quando la comunicazione è limitata ai thread all'interno dello stesso processo.
- **Funzioni di base:** sia i semafori con nome che quelli senza usano le classiche operazioni di semaforo:
 - `sem_wait()` : esegue l'operazione `wait()` (decrementa il conteggio e blocca se $S \leq 0$)
 - `sem_post()` : esegue l'operazione `signal()` (incrementa il conteggio e sblocca un thread in attesa).

Variabili condizionali in POSIX

Le variabili condizionali Pthreads sono utilizzate per attendere che una specifica condizione sui dati condivisi diventi vera.

- **Associazione necessaria:** a differenza dei monitor (che hanno il lock implicito), le variabili condizionali Pthreads devono essere sempre utilizzate in associazione con un lock mutex esplicito, che fornisce l'esclusione mutua per proteggere i dati condivisi.
- **Attesa:** la funzione `pthread_cond_wait()` esegue due azioni fondamentali:
 1. Rilascia atomicamente il mutex associato.
 2. Sospende il thread chiamante finché non viene segnalato.
- **Segnalazione:** la funzione `pthread_cond_signal()` è usata da un thread che ha modificato i dati per risvegliare un thread in attesa. Il thread che esegue la `signal()` non rilascia immediatamente il mutex; lo fa solo con la successiva chiamata `pthread_mutex_unlock()`.
- **Ripresa:** il thread risvegliato riacquisisce automaticamente il mutex prima di riprendere l'esecuzione.